

Borla

Technical Debt Plan

Student: **George Boadu Junior**

Student ID: **22427354**

Course: CSCD 602 — Advanced Software Engineering, University of Ghana

Document: Technical_Debt_Plan.pdf · Version 1.0 · 13 August 2026

1. How debt was tracked during the build

Every deliberate shortcut was written down **as the decision was made**, not reconstructed afterwards — most of the items below are quoted almost verbatim from code comments left at the exact line where the trade-off was taken (e.g. `server/src/jobs/index.ts`, `server/src/modules/auth/routes.ts`). This is the practice the course asks for: technical debt as a first-class, visible decision, not a silent shortcut discovered later.

Each item below follows **Debt** → **Cause** → **Impact** → **Priority** → **Proposed Resolution**, and is labelled as one of:

- **Acceptable temporarily** — correct trade-off for a graded demo at pilot scale; revisit only if the project continues past this exam.
- **Scheduled for future resolution** — fine for now, but must be fixed before real users or a meaningful scale-up.
- **Critical** — must not reach real users in its current form.

2. Debt register

ID	Debt	Cause	Impact	Priority	Proposed resolution
TD-01	No AI key by default — mostly resolved: a real <code>GEMINI_API_KEY</code> is now set on Render, so moderation actually calls Gemini instead of falling straight to the manual queue. A second, related defect (D-o8) surfaced immediately once the key was live: the hardcoded model ID (<code>gemini-2.5-flash</code>) started returning <code>404</code> — no longer available to new users against this key's account tier, so every call silently fell back to fail-closed/manual — fixed by switching to the <code>gemini-flash-latest</code> alias (<code>server/src/ai/moderation.ts</code>).	No Google AI Studio account was provisioned for the <i>original</i> build; the moderation pipeline (<code>server/src/ai/moderation.ts</code>) checks <code>GEMINI_API_KEY</code> and no-ops if absent — that fail-closed behaviour is exactly why D-o8 degraded silently to the manual queue instead of breaking the review flow.	Residual risk is now just "hasn't been re-verified live since the model-ID fix," not "no AI screening exists."	● Scheduled (down from unresolved — key is provisioned; pending one live moderation call against production to confirm <code>gemini-flash-latest</code> actually classifies correctly post-redploy)	Re-run a live moderation test (submit a borderline review in production, confirm a real <code>allow / flag / block</code> verdict instead of manual-queue fallback) once the model-ID fix is deployed.
TD-02	No real SMS gateway — mostly resolved: <code>server/src/utlils/sms.ts</code> sends the OTP via GiantSMS when <code>GIANTSMS_API_TOKEN</code> / <code>GIANTSMS_SENDER_ID</code> are configured, falling back to the in-app <code>devOtp</code> display only if the send fails or isn't configured. Confirmed live in production: <code>POST /api/auth/otp/request</code> against <code>https://borla.onrender.com</code> returned <code>{"delivered":true}</code> — GiantSMS accepted the reconstructed request shape (auth header, <code>from / to / msg body</code>) for real, not just in theory. A related gap closed at the same time: the two seeded demo numbers are arbitrary, not real handsets, so once real SMS was wired up they would have started reporting <code>delivered:true</code> while the code vanished into a gateway with no phone behind it — a graded demo would have been locked out. Fixed by special-casing <code>DEMO_PHONES</code> (<code>server/src/seed.ts</code>) to always use a fixed, documented code (<code>DEMO_OTP</code>) and never attempt a real send, regardless of gateway state.	The HTTP contract was reconstructed from published third-party client libraries, not a first-party OpenAPI spec. Production accepted the request; actual handset delivery to a <i>real</i> number has still not been visually confirmed (by definition, the two numbers this build can freely test against are the exempted demo ones).	Low residual risk: the gateway accepting the request end-to-end (not just returning a generic 200) is strong evidence the integration is correct for a genuine phone number, but "accepted by the gateway" and "arrived on a phone" are not identical.	● Scheduled (down from Critical — a real send now provably works at the HTTP layer, and grading no longer depends on it working at all)	Confirm one delivery to a real handset, then remove the <code>devOtp</code> fallback field for non-demo numbers entirely so a failed send surfaces as a retryable error instead of a security bypass.
TD-03	Native background geolocation was not built. Collectors only report position while their browser tab is open and in the foreground (<code>useGeolocation watchPosition</code> , no Capacitor/native service).	Requires an Android toolchain, a background-location plugin (commercial or complex to configure), and a physical/emulated device to validate "screen-off roaming" — none available, and a native APK isn't gradable as a "Live Application URL" per the exam's own submission format.	This is the single biggest gap from the original design's own risk register (<code>borla-technical-design.md</code> §14, risk #1) — a collector who backgrounds the app or locks their phone stops updating position, which is exactly the failure mode that erodes trust in the map.	● Critical (for any real pilot) / ● Acceptable for this exam	Wrap the existing shared React screens in Capacitor, add <code>@capacitor-community/background-geolocation</code> , and test on real low-end Android hardware (Tecno/Infinix/itel) per the original risk mitigation — no server-side change needed, since the API already accepts position updates from any authenticated client.
TD-04	No offline PWA shell — mostly resolved: <code>vite-plugin-pwa</code> now ships a real installable PWA — manifest + icons (<code>client/public/icon-*.png</code>), a Workbox service worker preconfiging the app shell (12 entries, ~490KB), an explicit "Update available" banner (<code>registerType: 'prompt'</code> , <code>src/pwa/UpdatePrompt.tsx</code> — never silently swaps content under an open tab) and a real	The remaining gap is genuinely more work (background sync / an offline action queue), not a missing dependency.	Households still see a blank state for anything requiring a live fetch while offline (map data, requests, broadcasts) — the shell now loads, the <i>data</i> doesn't.	● Acceptable (down from Scheduled — the install/update half is done; only the	Add a Workbox Background Sync queue (or a hand-rolled IndexedDB outbox) for "clear pin" and similarly small mutating actions so they survive a dropped connection.

ID	Debt	Cause	Impact	Priority	Proposed resolution
	"Install app" button (<code>src/pwa/InstallButton.tsx</code> , <code>beforeinstallprompt</code>). Verified: the service worker registers and activates (checked via a headless-browser <code>navigator.serviceWorker.getRegistrations()</code> call against the built <code>dist/</code>), and API/socket traffic is deliberately excluded from precaching so it's never served stale. Still open: households' geolocation is on-demand/foreground-only (unchanged), and there's no queued-writes-while-offline behaviour yet (e.g. "clear pin" typed while offline doesn't queue and flush on reconnect — the shell loads offline, but mutating actions still need a live connection).			offline-writes half remains)	
TD-05	No Redis/BullMQ — resolved. A real Upstash Redis instance now backs exactly what the original design specified: <code>presence:{id} / geo:collectors / heartbeat:zset</code> for live presence (<code>server/src/redis/presence.ts</code>), <code>pins:active</code> for broadcast pins, <code>ratelimit:otp:{phone} / ratelimit:notif:{id}</code> for rate limiting (<code>server/src/redis/rateLimit.ts</code>), a <code>Socket.IO</code> Redis adapter (<code>@socket.io/redis-adapter</code>) for cross-instance event delivery, and BullMQ replacing all four <code>node-cron</code> sweeps plus two new jobs — <code>fanout</code> (broadcast candidate-matching/notification, off the request thread with retries/backoff) and <code>moderate</code> (the Gemini call, same). Postgres/PostGIS stays the durable mirror, exactly per the original hot-path/cold-path split.	— (closed)	— (closed)	🟢 Resolved	Verified end-to-end against a real local Redis (Docker, mirroring the exact Upstash <code>rediss://</code> URL shape used in production): a collector going online is visible via <code>ZCARD geo:collectors / EXISTS presence:{id}</code> ; creating a broadcast registers <code>pins:active</code> synchronously and produces a real <code>broadcast_notifications</code> row via the async <code>fanout</code> job within ~1s; clearing the pin removes it from <code>pins:active</code> . 46 server tests passing (up from 39), including new coverage of the rate limiter and the async fan-out path. Production verification pending the user pasting <code>REDIS_URL</code> into Render's dashboard (<code>sync: false</code> in <code>render.yaml</code> , same pattern as the other secrets).
TD-06	Admin auth has no 2FA — phone + password only (<code>bcrypt</code> -hashed), same JWT mechanism as household/collector, instead of the original design's "stronger auth than the field apps."	Full 2FA (TOTP/SMS) needs either the same missing SMS gateway (TD-02) or a new TOTP flow — both extra scope for an account class that, in this build, is a single seeded demo user.	An admin account compromise is higher blast radius than a household/collector one (can suspend users, remove content) — acceptable only because there is exactly one demo admin account behind a strong generated password, never exposed publicly except in the graded credentials file.	🔴 Critical (before real ops staff use it)	Add TOTP (e.g. <code>otplib</code>) gated behind the existing <code>role='admin'</code> check; store a <code>totp_secret</code> column already anticipated by the schema's extensibility (JSONB <code>app_config</code> pattern could hold it, or a dedicated column).
TD-07	The standalone <code>admins</code> table from the original design was folded into <code>users.role = 'admin'</code> with a nullable <code>password_hash</code> column that only admins use.	Reduces schema surface for a single-admin demo; avoids a parallel user model.	Slight modelling smell — <code>password_hash</code> is <code>NULL</code> for 99% of rows (households/collectors) — and there is no dedicated <code>admins.role_tier</code> (<code>ops / moderator / superadmin</code> from the original design).	🟢 Acceptable	If multiple admin tiers are ever needed, split back into a dedicated <code>admins</code> table with its own role enum, migrated via a straightforward <code>INSERT ... SELECT from users WHERE role='admin'</code> .
TD-08	Leaflet + raw OpenStreetMap raster tiles instead of the original design's MapLibre GL + vector tiles.	Zero-config: no tile-provider account/key needed, which matters when nobody is provisioning new accounts mid-exam.	Slightly less crisp rendering and no offline vector caching; functionally equivalent for markers/popups/radius visualisation, which is all this build needs.	🟢 Acceptable	Swap the <code>MapView</code> component's tile layer for a MapLibre + vector source once a tile provider (e.g. MapTiler free tier) is chosen — the component's public interface (<code>center</code> , <code>points</code>) would not need to change.
TD-09	No i18n beyond English , despite the original design and the low-literacy design brief calling for Twi/Ga from day one.	Time-boxed; English-only was the fastest path to a fully functional, testable app across every screen.	Directly works against the stated accessibility goal for the target users.	🟡 Scheduled	The UI already isolates copy inside JSX rather than scattering it — introduce <code>react-i18next</code> (or the originally-planned <code>packages/shared/i18n</code>) and extract strings; start with Twi and Ga per the pilot-neighbourhood plan.
TD-10	Automated test coverage is broad, not deep. 46 server tests cover every route's happy path and its most important failure mode (idempotency guards, role gates, moderation gating, phone normalization, Redis rate limiting, async job side-effects); there is no fuzzing, no load testing, and client tests cover 2 of ~9 components.	A time-boxed exam favoured "every module has real integration tests" over "one module is exhaustively tested" — a deliberate breadth-over-depth call, consistent with §7.7 of the SRS.	Edge cases in less-tested paths (e.g. concurrent double-accept races under real network latency, malformed geo payloads) are not proven safe, only reasoned about.	🟡 Scheduled	Add property-based tests for the geospatial radius math, a concurrency test that fires <code>accept+reject</code> simultaneously against the same request to prove the conditional-update guard under real contention (not just sequential

ID	Debt	Cause	Impact	Priority	Proposed resolution
					calls), and React Testing Library coverage for <code>HouseholdHome / CollectorHome</code> .
TD-11	Photo → waste-type AI classification was not built (§18 Job B in the original design).	Explicitly deprioritised behind moderation (Job A) per the original design's own value ranking — moderation is the safety-critical one; photo classification is a UX nicety.	Households still pick a waste-type tile manually instead of snapping a photo — a real but non-blocking gap for low-literacy accessibility.	● Scheduled	Reuse the existing <code>aiClient</code> -shaped moderation module's pattern: a second thin wrapper calling Gemini's multimodal endpoint, wired to a new <code>POST /broadcasts image</code> field, client-side downscaled before upload.
TD-12	Provider call-masking was not built — accepted requests reveal each party's real phone number ("reveal-on-accept", the original design's own recommended v1 approach, §10 option 1).	This is the design's own recommended v1 trade-off, not a shortcut taken under exam pressure — kept as-is deliberately.	Real phone numbers are exchanged between two people who chose each other via an accepted request — acceptable given the low stakes described in the original design.	● Acceptable	Add provider number-masking (e.g. via Hubtel) only if abuse reports or scale warrant the added per-call cost, exactly as the original design already prescribes.
TD-13	Single Render Web Service, no staging environment, no CI pipeline beyond local <code>npm test</code> .	Free-tier, single-service deploy was the fastest path to a real "Live Application URL" within the exam window.	A bad commit reaches production directly; no automated gate before deploy.	● Scheduled	Add a GitHub Actions workflow running <code>npm test</code> on every push (config-only change, no app-code change needed) and a second Render environment for staging before promoting to the graded URL.

3. Debt NOT taken — deliberate design choices worth naming

To be explicit about the difference between *debt* (a shortcut that should eventually be paid down) and a *considered trade-off that stays*: the no-collector-binding broadcast plane, the absence of in-app payments, and reveal-on-accept-instead-of-provider-masking are not technical debt — they are the original design's own architecture, kept unchanged because they are still the right call at this scale (see `bor1a-technical-design.md` §1 and §10, and TD-12 above).

4. Repayment plan by phase

This maps directly onto `Project_Documentation.md` §15 (Maintenance & Future Evolution):

Phase	Items paid down
v1.1 (immediate post-submission, if continued)	TD-06 (admin 2FA) — the one remaining ● Critical item, must land before any real admin touches the system; confirm one real GiantSMS handset delivery to fully close TD-02
v1.2	TD-01 (re-verify live moderation post-model-fix — key is already provisioned), TD-13 (CI pipeline)
v1.3	TD-03 (native Capacitor collector app + background geolocation) — the single biggest lift, matched to the original design's own phased roadmap
v2.0	TD-09 (i18n), TD-04 (offline write queue — the install/update half already shipped), TD-11 (photo classification)
Ongoing	TD-10 (expand test depth every time a new bug class is found in production)